

CSS variables and organization

Once a stylesheet grows, one of the hard parts is keeping repeated decisions consistent:

- The same blue appears in links, buttons, labels, and borders.
- The same spacing values appear in cards, sections, and rows.
- The same radius appears on every panel.
- A component variant needs a different accent color without rewriting the whole component.

CSS variables help with that by giving a value a name, so you can reuse it and change it from one place.

Start with repeated values

Use a small dashboard where the same design choices appear in several places. The page works, but the CSS repeats the same colors, spacing, and radius values many times.

Start with this markup:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>CSS variables practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Keep repeated design values under control</h1>
        <p class="summary">Colors, spacing, and component variants become easier to manage</p>
      </section>

      <section class="course-grid" aria-label="Course cards">
        <article class="course-card">
          <p class="card-label">Core lesson</p>
          <h2>CSS selectors</h2>
          <p class="card-description">Review how selectors match elements and how the browser interprets them</p>
          <a class="card-action" href="/css/selectors">Open lesson</a>
        </article>

        <article class="course-card is-success">
          <p class="card-label">Project ready</p>
          <h2>Responsive page</h2>
          <p class="card-description">Use layout, media, and accessibility habits together</p>
        </article>
      </section>
    </main>
  </body>
</html>
```



```
    <a class="card-action" href="/css/project">Open project</a>
  </article>
</section>

<aside class="notice">
  <strong>Reminder:</strong> Keep class names meaningful and keep repeated design
</aside>
</main>
</body>
</html>
```

Now add this deliberately repetitive CSS:

CSS

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
```

```
.page {
  max-width: 960px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.course-card,
.notice {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.intro,
.course-grid {
  margin-bottom: 24px;
}

.eyebrow,
.card-label {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}

.summary,
.card-description {
  color: #4b5563;
  line-height: 1.6;
}

.course-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}

.course-card {
  border-top: 4px solid #2563eb;
```

```
}

.course-card.is-success {
  border-top-color: #059669;
}

.course-card.is-success .card-label {
  color: #059669;
}

.card-action {
  display: inline-block;
  margin-top: 16px;
  padding: 10px 14px;
  color: white;
  background-color: #2563eb;
  border-radius: 8px;
  font-weight: 700;
  text-decoration: none;
}

.notice {
  color: #1e3a8a;
  background-color: #dbeafe;
  border-color: #93c5fd;
}
```

Open the page. It should look like a small dashboard with two cards and a notice. Now look at the CSS as a maintenance problem.

The blue `#2563eb`, the radius `8px`, and the spacing values `16px` and `24px` all appear in several places.

If the brand color changes, you have to hunt through the stylesheet and replace the same value in multiple rules.

That is the problem CSS variables solve.

Give repeated values names

A CSS variable is written as a custom property. Custom property names start with two dashes:

CSS

```
:root {
  --color-brand: #2563eb;
}
```

`:root` selects the top-level element of the page. For normal HTML pages, you can think of it as the highest place to put values that should be available everywhere.

In this rule, `--color-brand` is the custom property name, and `#2563eb` is the value stored inside that name.

The name is yours, but it must start with `--`.

Now add a small set of variables at the top of the stylesheet:

CSS

```
:root {
  --color-text: #111827;
  --color-muted: #4b5563;
  --color-page: #f3f4f6;
  --color-surface: #ffffff;
  --color-border: #e5e7eb;
  --color-brand: #2563eb;
  --color-brand-soft: #dbeafe;
  --color-brand-border: #93c5fd;
  --color-success: #059669;
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --radius-card: 8px;
}
```

These names describe design decisions:

- `--color-brand` is the main brand color.
- `--color-muted` is for quieter text.
- `--space-md` is a reusable spacing value.
- `--radius-card` is the radius used on card-like surfaces.

The point is not to make a variable for every single value. The point is to name values that repeat or carry meaning.

💡 Name values by job - Prefer names like `--color-brand` , `--color-border` , and `--space-lg` over names like `--blue` or `--twenty-four` . A job-based name can survive a design change.

Use variables with `var()`

Defining a variable does not change the page by itself. To use the value, call it with `var(...)` .

Replace the first few repeated values:

CSS

```
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: var(--color-text);
  background-color: var(--color-page);
}

.intro,
.course-card,
.notice {
  background-color: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-card);
  padding: var(--space-lg);
}
```

`var(--color-text)` means "use the value stored in `--color-text`."

`var(--color-page)` means "use the value stored in `--color-page`."

The browser replaces each `var(...)` with the custom property value when it computes the final CSS.

Now the page should look the same, but the stylesheet is easier to change.

If you change `--color-page` in `:root`, every place using `var(--color-page)` updates.

Replace the repeated brand values

Now replace more repeated values with variables:

CSS

```
.intro,
.course-grid {
  margin-bottom: var(--space-lg);
}

.eyebrow,
.card-label {
  margin-top: 0;
  color: var(--color-brand);
  font-size: 14px;
  font-weight: 700;
}

.summary,
.card-description {
  color: var(--color-muted);
  line-height: 1.6;
}

.course-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: var(--space-md);
}

.course-card {
  border-top: 4px solid var(--color-brand);
}

.card-action {
  display: inline-block;
  margin-top: var(--space-md);
  padding: 10px 14px;
  color: white;
  background-color: var(--color-brand);
  border-radius: var(--radius-card);
  font-weight: 700;
  text-decoration: none;
}

.notice {
  color: #1e3a8a;
  background-color: var(--color-brand-soft);
  border-color: var(--color-brand-border);
}
```

The result should still look the same. That is an important point: variables do not have to change the design immediately. They make the design easier to maintain.

Try changing `--color-brand` from `#2563eb` to `#7c3aed`. The label, card border, and button should change together. That is the practical payoff.

Variables follow the cascade

CSS variables are still CSS, which means they follow the cascade and can be scoped to part of the page.

Right now, the success card needs a different accent color. The old CSS did this by styling each piece separately:

CSS

```
.course-card.is-success {
  border-top-color: #059669;
}

.course-card.is-success .card-label {
  color: #059669;
}
```

That works, but it repeats the idea: this card's accent should be green. Instead, replace those separate success-card rules with a local accent variable on each card:

CSS

```
.course-card {
  --card-accent: var(--color-brand);
  border-top: 4px solid var(--card-accent);
}

.course-card .card-label {
  color: var(--card-accent);
}

.course-card.is-success {
  --card-accent: var(--color-success);
}
```

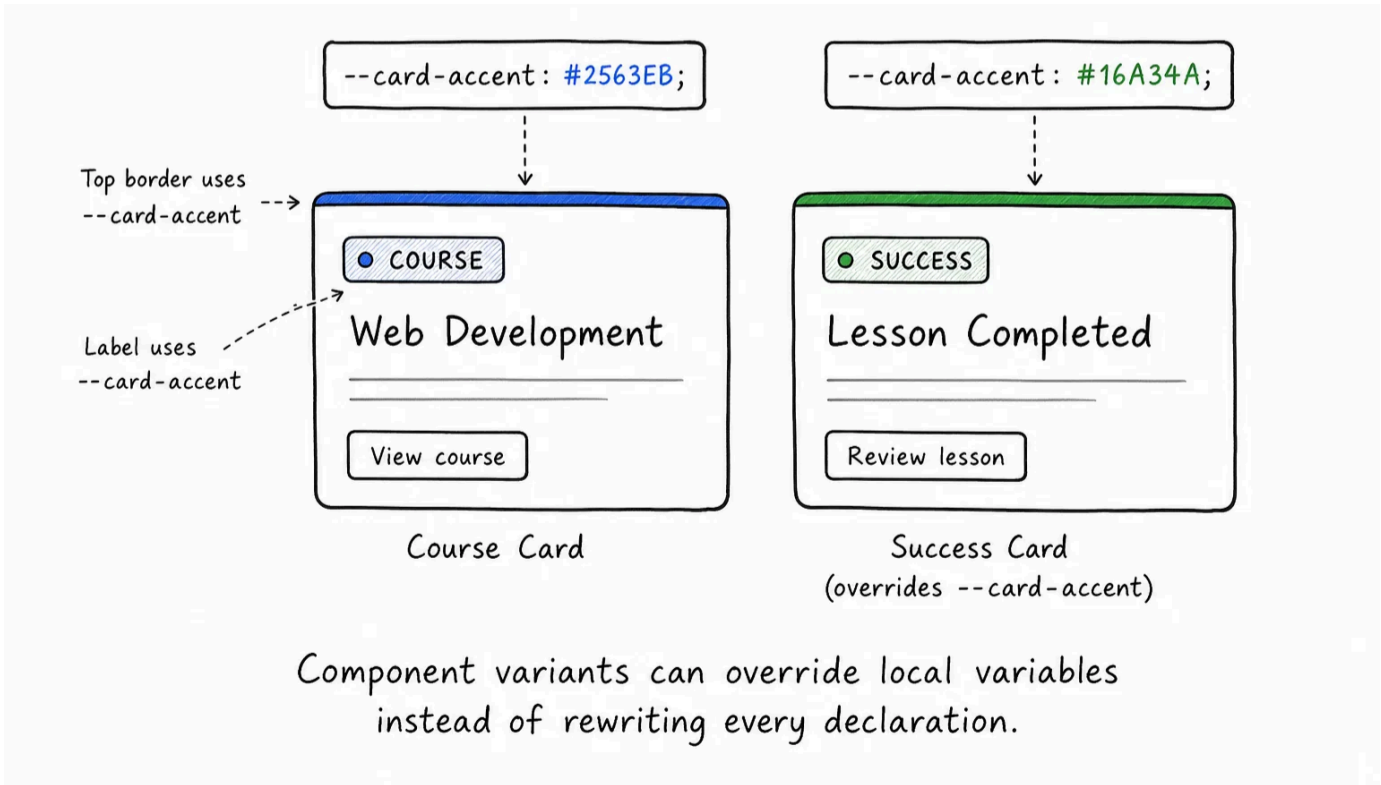
The normal card sets `--card-accent` to the brand color.

The card border and card label both use `var(--card-accent)`.

The success card overrides only `--card-accent`.

Because custom properties inherit, the `.card-label` inside the success card can use the green value from its parent card.

This is one of the best uses of CSS variables: define a component once, then change a small set of local values for variants.



Use fallbacks when a value might be missing

`var()` can include a fallback value. The fallback is used if the custom property is missing or invalid.

CSS

```
.card-action {  
  background-color: var(--color-action, #2563eb);  
}
```

This means: use `--color-action` if it exists. If it does not exist, use `#2563eb`.

Fallbacks are useful when:

- A component may be used on pages that do not define every variable.
- A theme may provide optional values.
- You want a safe default while the stylesheet is being refactored.

For this lesson, most values live in `:root`, so fallbacks are not required everywhere. Use them where a missing value would make the component hard to read or use.

Organize the stylesheet by job

Variables help values stay consistent, and organization helps people find the right rule later.

CSS comments use this syntax:

CSS

```
/* This is a CSS comment. */
```

The browser ignores comments because they are for people reading the file.

A small stylesheet can be organized like this:

CSS

```
/* 1. Design values */
:root {
  --color-text: #111827;
  --color-muted: #4b5563;
  --color-page: #f3f4f6;
  --color-surface: #ffffff;
  --color-border: #e5e7eb;
  --color-brand: #2563eb;
  --color-brand-soft: #dbeafe;
  --color-brand-border: #93c5fd;
  --color-success: #059669;
  --space-sm: 8px;
  --space-md: 16px;
  --space-lg: 24px;
  --radius-card: 8px;
}

/* 2. Base styles */
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: var(--color-text);
  background-color: var(--color-page);
}

/* 3. Layout */
.page {
  max-width: 960px;
  margin: 40px auto;
  padding: 0 var(--space-md);
}

.course-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: var(--space-md);
}

/* 4. Components */
.course-card {
  --card-accent: var(--color-brand);
  background-color: var(--color-surface);
  border: 1px solid var(--color-border);
  border-top: 4px solid var(--card-accent);
  border-radius: var(--radius-card);
  padding: var(--space-lg);
}

/* 5. Variants */
.course-card.is-success {
  --card-accent: var(--color-success);
}
```

The exact section names can change by project. The useful habit is to group rules by job:

- Design values first.
- Base element styles next.
- Layout rules near other layout rules.
- Component rules near the component.
- Variants and state rules close enough that they are easy to find.

Do not over-organize a tiny file. But once a stylesheet grows, sections help you avoid hunting.

Do not turn every value into a variable

CSS variables are useful, but using too many can make CSS harder to read. This is probably not worth a variable:

CSS

```
.card-action {  
  padding: 10px 14px;  
}
```

The two values are specific to this button shape. If they are not reused elsewhere, leaving them directly in the rule is fine.

This is more likely worth a variable:

CSS

```
:root {  
  --radius-card: 8px;  
}
```

The same radius appears on cards, notices, and buttons. If the design changes from `8px` to `10px`, one variable can update the shared shape.

Use variables for repeated values, meaningful design decisions, theme values, and component variants.

Leave one-off details alone until they actually need a name.

 **Variables do not fix unclear design decisions** - A variable name should make the CSS easier to understand. If `--thing`, `--blue-2`, or `--random-space` makes you stop and guess, the variable is not helping yet.

Inspect variables in DevTools

DevTools can show how variables resolve. Inspect the normal card and look at the `border-top` declaration; you should see the browser resolving `var(--card-accent)` to the brand color.

Then inspect the success card. The same `border-top` declaration should resolve to the success color because `.course-card.is-success` overrides `--card-accent`.

Try changing `--color-brand` in DevTools, and every declaration using `var(--color-brand)` should update.

If a variable does not work, check these things:

- Does the name start with `--` when you define it?
- Does the `var(...)` call use the exact same name?
- Is the variable defined on this element or inherited from an ancestor?
- Is another rule overriding the variable later in the cascade?
- Does `var(...)` need a fallback?

Variables are easier to debug when you remember that they still follow normal CSS rules.

CSS variable map

Here is the practical map:

Pattern	What it does
<code>--color-brand: #2563eb;</code>	Defines a custom property.
<code>var(--color-brand)</code>	Uses the custom property value.
<code>var(--color-brand, #2563eb)</code>	Uses a fallback if the custom property is missi
<code>:root { ... }</code>	Stores values that should be available across t
<code>.course-card { --card-accent: ... }</code>	Creates a local component value.
<code>.is-success { --card-accent: ... }</code>	Overrides the local value for a variant.

Try it

Use the same dashboard and test the variables directly:

- Change `--color-brand` in `:root` . Notice which parts of the page update together.
- Change `--radius-card` from `8px` to `16px` . Notice how shared rounded corners change.
- Remove `--card-accent` from `.course-card` . Check what happens to the card border and card label.
- Change `.course-card.is-success` to set `--card-accent: #dc2626;` . Notice that the success card variant changes without rewriting the border

and label rules.

- Add a fallback to one `var()` call, then temporarily misspell the variable name. Confirm that the fallback is used.
- Add a second `.course-card.is-success` rule later in the stylesheet with a different `--card-accent`. Notice that the later matching variable declaration wins.
- Add one new variable only if a value repeats at least twice. Leave one-off values alone.

CSS variables are most useful when they make change safer. If a value has a clear job and appears in several places, give it a name. If it is a one-time detail, keep the CSS direct.

< 0 / 10 >

✔ Knew 0 Items

✧ Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is a CSS variable?

[Click to Reveal the Answer](#)

✔ Already Know that

✧ Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →